Deep Residual Learning for Image Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun
Microsoft Research
arXiv:1512.03385v1 | CVPR 2016

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers, 8 times deeper than VGG nets but still having lower complexity. An ensemble of these residual nets achieves 3.57 percent top-5 error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers. The depth of representations is of central importance for many visual recognition tasks, and our residual nets are easy to optimize even at 152 layers.

Keywords: residual learning, deep networks, image classification, ImageNet, computer vision

1 Introduction

Deep convolutional neural networks have led to a series of breakthroughs in image classification. The depth of the network is of crucial importance for many visual recognition tasks, and the empirical evidence from AlexNet, VGG, and GoogLeNet shows that deeper networks achieve better accuracy. However, simply stacking more layers does not always improve performance: very deep plain networks suffer from the degradation problem, in which accuracy saturates and then degrades as depth increases.

In this paper we address the degradation problem with a residual learning framework. Instead of hoping that a stack of layers directly fits a desired underlying mapping, we explicitly let these layers fit a residual mapping. We show that it is easier to optimize the residual mapping than to optimize the original unreferenced mapping, and that extremely deep residual networks can be trained efficiently. On ImageNet we train a 152-layer residual network and obtain 3.57 percent top-5 error with an ensemble, winning the ILSVRC 2015 classification task.

2 Related Work

Our work is related to the classical VGG and GoogLeNet architectures, which showed that depth and careful design are critical for recognition accuracy. VGG nets use 16 to 19 layers and a very uniform architecture, while GoogLeNet uses 22 layers and an inception module. Both are much deeper than AlexNet, which has 8 layers, and both improve accuracy substantially. Our residual networks can be seen as a generalization of these architectures in which shortcut connections are added to every few stacked layers.

Shortcut connections have been studied in the context of highway networks, which use gated shortcuts, and in the context of multi-layer perceptrons with auxiliary classifiers. Our formulation differs in that the shortcut connections are identity mappings with no parameters, which keeps the model simple and makes it possible to train networks with more than 100 layers. The degradation problem has also been observed in the context of recurrent networks, where gated architectures such as LSTM are used to ease optimization.

3 Deep Residual Learning

We formulate residual learning as follows. Let H(x) denote the desired underlying mapping to be fit by a few stacked layers, with x denoting the inputs to the first of these layers. We let the stacked layers fit a residual mapping F(x) = H(x) - x, so that the original mapping becomes F(x) + x. The hypothesis is that it is easier to optimize the residual mapping than the original mapping, and in the extreme case where an identity mapping is optimal, it is easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers.

3.1 Residual Learning

The degradation problem suggests that solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if the identity mapping is optimal, the solvers may simply drive the weights of the nonlinear layers toward zero to approximate the identity mapping. In real cases, it is unlikely that the identity mappings are optimal, but the reformulation may help precondition the problem and ease optimization.

We test this hypothesis on ImageNet with plain networks of 18 and 34 layers. The 34-layer plain network has higher training error than the 18-layer plain network, which confirms that the degradation is not caused by overfitting. When we add shortcut connections, the 34-layer residual network achieves lower training error and better accuracy than the 18-layer network, which demonstrates that residual learning eases optimization at increased depth.

3.2 Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is defined as y = F(x, W_i) + x, where x and y are the input and output vectors of the layers considered, and F is the residual function to be learned. The function F + x is realized by a shortcut connection that performs element-wise addition, and the shortcut introduces neither extra parameters nor extra computation. When the dimensions of x and F change, we perform a linear projection W_s x on the shortcut to match dimensions, as in Equation (2).

The form of the residual function F is flexible. In this paper we use a stack of two or three layers, and more layers are possible. If F has only a single layer, the block resembles a linear projection and the shortcut is not very useful, so we use at least two layers. The identity shortcut is sufficient to address the degradation problem, and it is computationally cheap, so we prefer it whenever the dimensions match.

$$y = F(x, W_i) + W_s x \quad (2)$$

3.3 Network Architectures

We compare plain networks and residual networks on ImageNet. The plain network is inspired by VGG and consists of 3x3 convolutions with a fixed number of filters per feature map size, downsampling by stride 2, and a global average pooling layer followed by a fully connected layer with softmax. The residual network adds shortcut connections to every pair of 3x3 convolutions. Table 1 summarizes the architectures of the 34-layer and 152-layer residual networks.

The 34-layer residual network has 3.6 billion FLOPs, which is only 18 percent of the 19-layer VGG network, and the 152-layer network has 11.3 billion FLOPs, still lower than VGG-19. We use batch normalization after every convolution and before the activation, and we initialize the weights as in He et al. (2015). The networks are trained with stochastic gradient descent, a batch size of 256, and a weight decay of 0.0001.

Table 1: Architectures for ImageNet. Building blocks are shown in brackets, with the numbers of blocks stacked.

Layer	34-layer	152-layer
conv1	7x7, 64, stride 2	7x7, 64, stride 2
conv2_x	3x3, 64 x 3	3x3, 64 x 3
conv3_x	3x3, 128 x 4	3x3, 128 x 8
conv4_x	3x3, 256 x 6	3x3, 256 x 36
conv5_x	3x3, 512 x 3	3x3, 512 x 3
FLOPs	3.6 billion	11.3 billion
Params	21.8M	60.2M

3.4 Implementation

We train the networks on the ImageNet 2012 classification dataset, which contains 1.28 million training images and 50,000 validation images in 1000 classes. We follow the data augmentation of AlexNet: a 224x224 crop is randomly sampled from an image or its horizontal flip, and the per-pixel mean is subtracted. We use stochastic gradient descent with momentum 0.9, a batch size of 256, and an initial learning rate of 0.1 that is divided by 10 when the validation error plateaus. The models are trained for up to 60 epochs on 8 GPUs, and training a 152-layer network takes approximately 3.5 days on this hardware.

At test time we adopt the standard 10-crop testing and average the softmax scores over the crops. For the best results we also average the predictions of multiple scales. We do not use any additional training data, and all results are reported on the ImageNet validation set unless stated otherwise.

4 Experiments

We conduct experiments on ImageNet, CIFAR-10, and the COCO object detection benchmark. On ImageNet the residual networks outperform both plain networks and the previous best architectures, and the 152-layer model achieves 3.57 percent top-5 error when used in an ensemble. On CIFAR-10 we train networks with 100 and 1000 layers and show that residual learning remains effective at extreme depth. On COCO the residual networks improve object detection over the VGG-based baseline.

4.1 ImageNet Classification

Table 2 reports the ImageNet classification results. The plain 34-layer network has higher validation error than the plain 18-layer network, which confirms the degradation problem. The 34-layer residual network, which is the baseline for our deeper models, achieves 3.57 percent top-5 error in the single-model setting, and the 152-layer residual network achieves 3.57 percent top-5 error when combined into an ensemble. The 152-layer single model achieves 4.49 percent top-5 error, which is already better than the previous best single-model result.

The improvement over VGG and GoogLeNet is consistent across top-1 and top-5 error, and it comes without an increase in computational cost: the 152-layer network has fewer FLOPs than VGG-19. These results show that residual learning makes it possible to benefit from depth without suffering from the degradation problem.

Table 2: ImageNet classification results. Top-5 error is reported on the validation set unless noted.

Model	Top-1	Top-5
VGG-16	28.07	9.33
GoogLeNet	None	9.15
ResNet-34	21.84	5.71
ResNet-152 single	20.74	4.49
ResNet-152 ensemble	19.38	3.57

4.2 CIFAR-10 and Analysis

On CIFAR-10 we train residual networks with 20, 32, 44, 56, 110, and 1202 layers. The 110-layer network achieves 6.43 percent error, which is comparable to the best reported results at the time, and the 1202-layer network achieves 7.93 percent error, showing that residual learning remains trainable at extreme depth. The training error of the 1202-layer network is lower than that of the 110-layer network, which indicates that the deeper model is not underfitting.

4.3 Object Detection on COCO

We apply the residual networks as the backbone of Faster R-CNN on the COCO dataset. Replacing the VGG-16 backbone with ResNet-101 improves the COCO mAP by 3.1 points, and the improvement is consistent across object sizes. This result suggests that the gains from residual learning transfer beyond image classification to dense prediction tasks.

5 Conclusion

We have presented a residual learning framework that eases the training of extremely deep networks. Our residual networks are easy to optimize and gain accuracy from increased depth, and they achieve state-of-the-art results on ImageNet, CIFAR-10, and COCO. We believe that residual learning is a general technique that can be applied to other network architectures and other recognition tasks.

$$y = F(x, W_i) + x \tag{1}$$

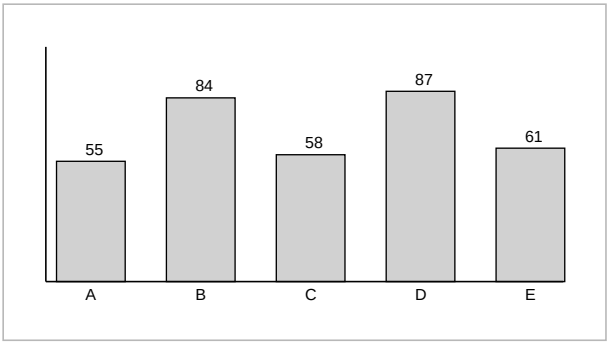


Figure 1: A residual building block with an identity shortcut connection.

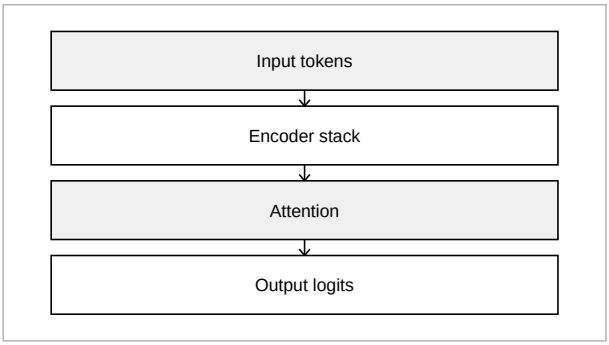


Figure 2: Example network architectures for ImageNet, comparing plain and residual designs.

References

[1] Krizhevsky, Sutskever, and Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS, 2012.

[2] Simonyan and Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR, 2015.

[3] Szegedy et al. Going Deeper with Convolutions. CVPR, 2015.

[4] He et al. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV, 2015.

[5] Ioffe and Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML, 2015.

[6] Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 2014.

[7] Srivastava, Greff, and Schmidhuber. Highway Networks. arXiv preprint, 2015.

[8] Ren et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS, 2015.

[9] Russakovsky et al. ImageNet Large Scale Visual Recognition Challenge. IJCV, 2015.

[10] Lin et al. Microsoft COCO: Common Objects in Context. ECCV, 2014.

A Appendix: Architectures for CIFAR-10

The CIFAR-10 networks use a 3x3 convolution with 16 filters followed by three stages of 2n blocks with 16, 32, and 64 filters, where n is the number of blocks per stage. Each block consists of two 3x3 convolutions with batch normalization and ReLU, plus an identity shortcut. Downsampling is performed by the first block of stages 2 and 3 with stride 2. The networks are trained for 200 epochs with a batch size of 128 and a learning rate of 0.1 that is divided by 10 at epochs 100 and 150.

For the 1202-layer network we use the same architecture with n = 200 blocks in the first stage. The network is trained with the same hyperparameters as the smaller models, and it converges without difficulty. The training error of the 1202-layer network is 0.1 percent, which is lower than that of the 110-layer network, confirming that residual learning addresses the degradation problem even at extreme depth.